

Urban Mobility Data Explorer — Technical Report

Team BroCode | African Leadership University | June 2026

Section 1: Problem Framing & Dataset Analysis

Dataset Description

The Urban Mobility Data Explorer (UMDE) is based on three official datasets provided by the New York City Taxi & Limousine Commission (TLC). The main data source is the yellow taxis trips data for the month of January 2026 in the form of a parquet file. This fact table consists of 3,724,889 raw trip records for pickup and dropoff dates, trip distances, fare structure details and location identifiers. There are also two supporting datasets: the taxi zone lookup CSV (265 zone-to-borough mappings) and the taxi zones GeoJSON file (geographic polygon boundaries for each zone).

The data is authentic operational data on urban mobility, covering all yellow taxi trips made in NYC within one month, at the transaction level. This provides good opportunities for the analysis of spatial demand distribution, fare economics and urban movement patterns.

Data Challenges

When loading and checking the raw parquet file, the following major data quality concerns were identified:

The greatest issue, as noted by the majority of the 1,202,764 records that were deleted during cleaning (roughly 32% of the raw data), was missing values. There were a number of columns with null values in a large batch of records, such as **Airport_fee** and **RatecodeID**. It was decided not to use impute because the loss of rows with missing data in any of the 10 selected columns would not likely impact the analysis, as the drop would not create artificial records that could influence the results.

The second most common set of removed records were logical outliers: when the fare was 0 or negative, when the trip distance was 0, when the total fare was 0, or when the dropoff timestamp was earlier than the pickup timestamp. These are either an entry error or trips that were cancelled but not erased.

A lesser but significant group were the extreme outliers, those trips that were more than 100 miles away or the fares over \$500. The ones were removed because they are not within the reasonable duration of an average yellow taxi trip in New York City.

Cleaning Assumptions

Every decision to exclude records was reported in `exclusion_log.csv`, where you can find the number of records, reason, and time of exclusion for each batch of otherwise included records. Throughout it was presumed that clean data should be physically and economically admissible for completed taxi trips within the five boroughs and the two main airports.

Unexpected Observation

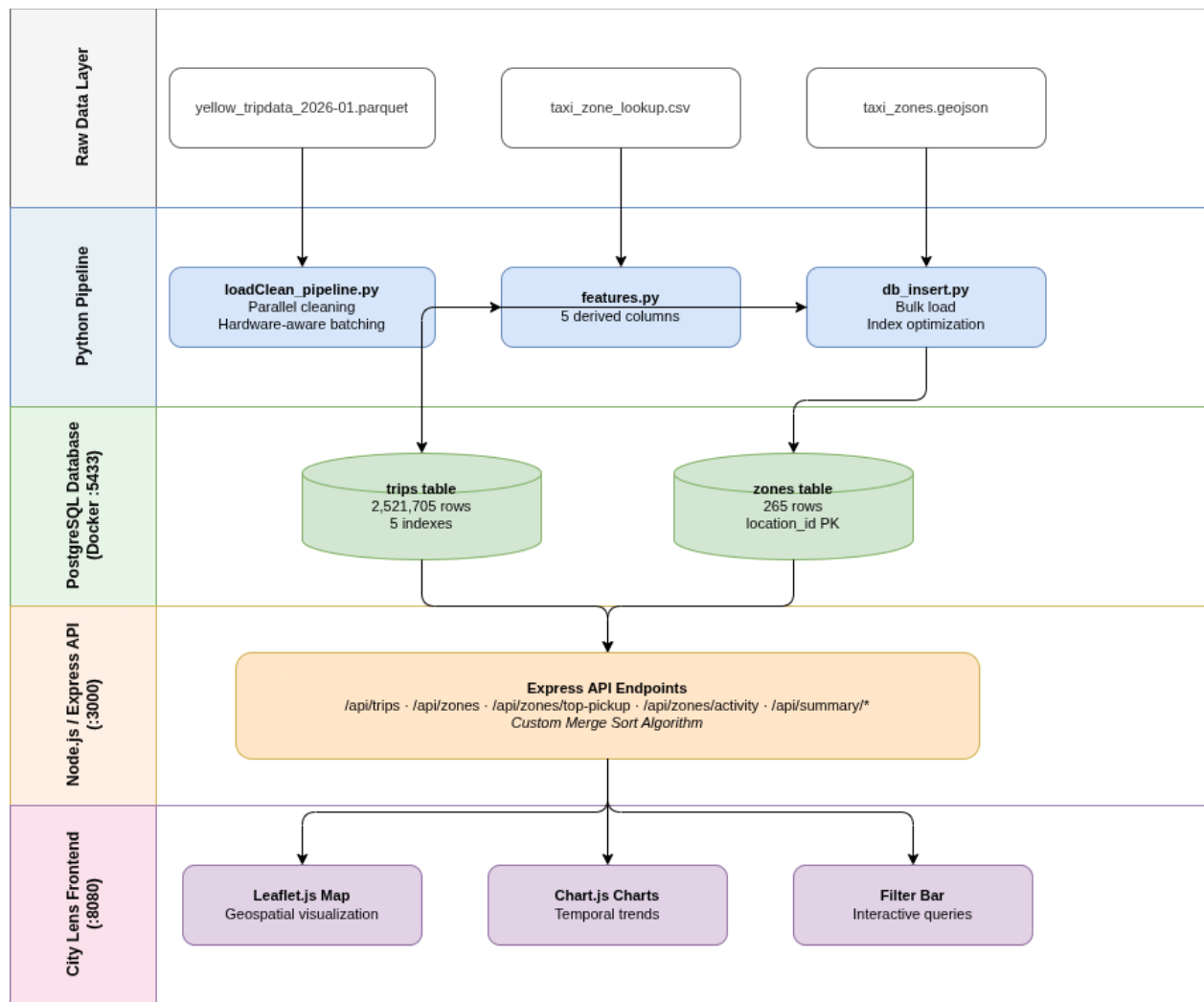
One of the most surprising pieces of information that was found when exploring the data was the distribution of missing data. The gaps were not scattered across rows, but clustered into batches, implying that there was not a single data entry error, but that the data was missing from certain vendor IDs or time periods. This was taken into account when designing the pipeline, employing a multiprocessing module from the python programming language, which enabled to process the data in parallel batches and allow each to be cleaned and logged without influencing the work of another batch.

Section 2: System Architecture & Design Decisions

Architecture Overview

The system is based on four layers:

- **Layer 1 - Raw Data:** Three source files (parquet, CSV, GeoJSON) in `data/raw/tripData/` converted into a layer. All parquet files found in this folder are automatically loaded by the pipeline without requiring any code modifications; thus, additional months of data can be added without writing any code.
- **Layer 2 - Python Pipeline:** There are three sequential scripts for the data engineering. `loadClean_pipeline.py` loads parquet files in chunks and cleans data in parallel with multiprocessing; `features.py` generates five new columns from the cleaned data; and `db_insert.py` bulk-loads the enriched data to Postgres with optimized loading, disabling indexes during the load and enabling them back up.
- **Layer 3 - PostgreSQL Database:** A normalized relational schema with two tables (zones, trips) in a Docker container on port 5433. The `pg` library is used to connect to the postgres database, and it is done through a connection pool with Node.js backend using environment variables from `.env`.
- **Layer 4 - Node.js/Express API + Frontend:** An Express server provides six REST endpoints which the frontend calls to fetch real data. All the data is dynamically loaded in the backend (API), and then displayed in the frontend (HTML/CSS/JavaScript), which features a Leaflet.js map and Chart.js charts.



Stack Justification

Python for the pipeline: Parquet is natively supported by PyArrow and pandas, so the data engineering language of choice is Python. There is no similar tooling available for reliable processing of parquet at scale in Node.js.

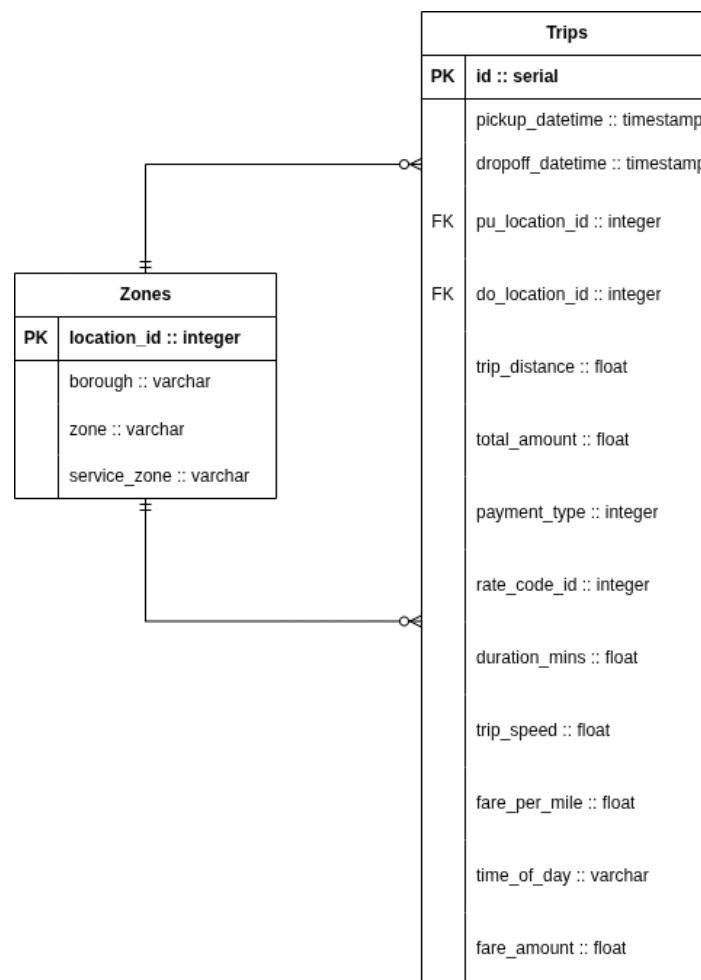
API layer: Stateless query and response logic using Node/Express is ideal for handling the API layer. Because of the connection pool to serve concurrent requests, Express can serve several concurrent requests from the front without blocking.

PostgreSQL via Docker: Postgres chosen over SQLite for size (2.5mil rows), and concurrency needs. The beauty of Docker is that there are no installation problems with environment-specific settings, the database can be spun up with one command on any machine that has Docker installed.

Schema Design

Schema pattern is star schema – zones as dimension table, trips as the fact table. Foreign keys to both zones.pu_location_id and zones.do_location_id are defined for trips, and the latter is also joined to zones for trips do_location_id to provide geographic data, so that JOIN queries can populate the trips table with the names of their boroughs and zones without providing that information in every trip row.

There are 5 indexes: pickup_datetime, pu_location_id, do_location_id, time_of_day, fare_amount. These indexes are dropped prior to bulk data insertion and then rebuilt after; this is deliberate because the cost of rebuilding the indexes outweighed the cost of bulk data insertion and reduced the bulk data insertion time from an estimated 25-40 minutes to an estimated insertion time of 8 minutes for 2.5 million rows.



Trade-offs

The choice of one month data instead of 2025 full year data was intentional, as the analysis would become too lengthy if information were collected over the entire year, and the system would become too cumbersome to operate. The entire year would have taken around 45 million rows, a longer run time of the pipeline, and a dump too large to submit to the database. January 2026 has 2.5 million clean records, enough to give statistically meaningful insights across all boroughs and across time.

Section 3: Algorithmic Logic & Data Structures

Problem Statement

The system should take in 265 taxi zones, each with a number of taxi trips, and return the top N taxi zones by number of taxi trips they had. The naive approach is to leave this responsibility to a SQL ORDER BY, but this requires that you do not examine, explain or credit algorithmic work that occurs in the database.

Combine Sort: Merge Sort

In `UMDE-BE/src/utlis/algorithm.js` a merge sort algorithm was manually written to sort zones based on the number of trips, sorted in descending order. The implementation doesn't rely on the built-in `.sort()` method of JavaScript, the `Array.prototype.sort` comparator, nor any third-party sorting library.

The `GET /api/zones/top-pickup` endpoint uses the algorithm: It returns all 265 zones in the database, which are not sorted, and the JavaScript merge sort sorts them, then it slices away the top N zones and returns them to the frontend.

Implementation

```
31 // Merge two sorted arrays into one sorted array
32 function merge(left, right, key) {
33   const result = [];
34   let i = 0;
35   let j = 0;
36
37   while (i < left.length && j < right.length) {
38     // Descending order - higher count comes first
39     if (left[i][key] >= right[j][key]) {
40       result.push(left[i]);
41       i++;
42     } else {
43       result.push(right[j]);
44       j++;
45     }
46   }
47
48   // Append remaining elements
49   while (i < left.length) {
50     result.push(left[i]);
51     i++;
52   }
53   while (j < right.length) {
54     result.push(right[j]);
55     j++;
56   }
57
58   return result;
59 }
60
61 // Recursively split and merge
62 function mergeSort(arr, key) {
63   // Base case - single element is already sorted
64   if (arr.length == 1) return arr;
65
66   // Split array in half
67   const mid = Math.floor(arr.length / 2);
68   const left = mergeSort(arr.slice(0, mid), key);
69   const right = mergeSort(arr.slice(mid), key);
70
71   return merge(left, right, key);
72 }
73
74 /**
75  * Rank zones by trip count using custom merge sort
76  * @param {Array} zones - array of zone objects with trip count
77  * @param {number} topN - how many top zones to return
78  * @returns {Array} sorted zones
79  */
80 function rankZonesByTripCount(zones, topN = 10) {
81   // Convert trip count to integer first
82   const parsed = zones.map(z => ({
83     ...z,
84     trip_count: parseInt(z.trip_count)
85   }));
86
87   const sorted = mergeSort(parsed, 'trip_count');
88   return sorted.slice(0, topN);
89 }
90
91 module.exports = { rankZonesByTripCount, mergeSort };
```

Complexity Analysis

Time Complexity: $O(n \log n)$ This is done because the array is halved with each recursive call, so there are $\log n$ recursive calls. Each level, n elements are processed in the merge step. The total number of comparisons then is $n \times \log n$.

Space Complexity: $O(n)$ Space is being used for the new array at each merge step. During the entire recursion the total memory used by all calls of the merge is proportional to n .

Justification

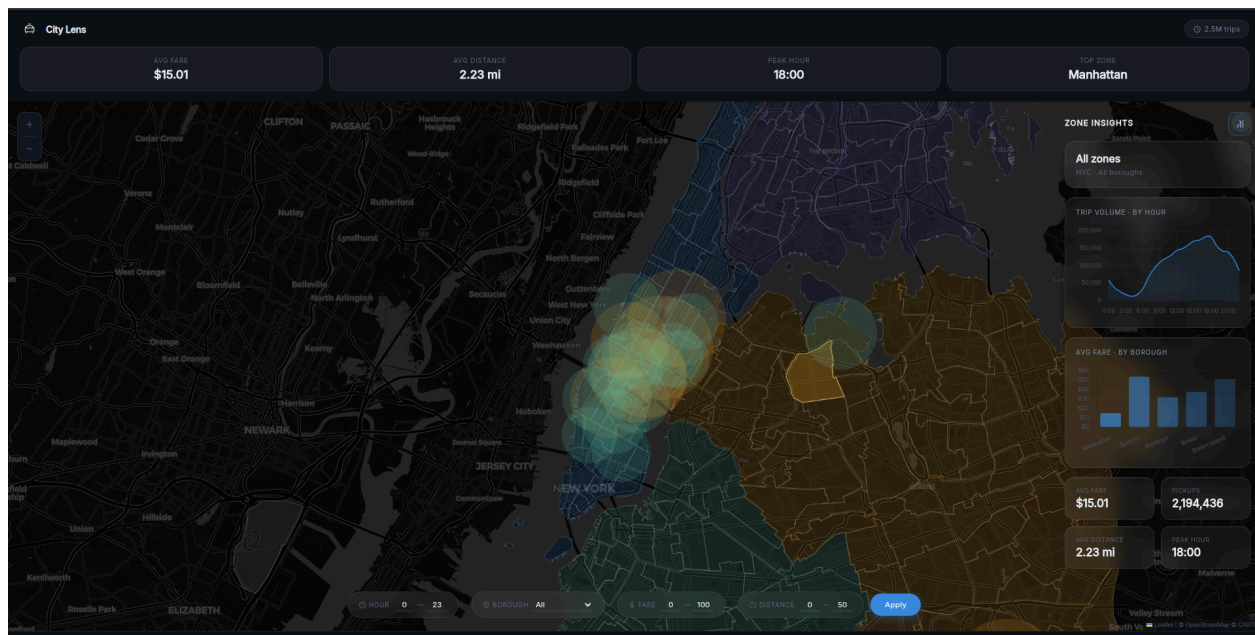
The reason why merge sort was chosen over other sorting methods such as insertion sort and bubble sort is that it has $O(n \log n)$ worst case performance. The difference in practice is minimal - but the algorithm shows “divide and conquer” thinking and yields a “stable sort,” in which zones with the same number of trips keep their relative order.

Section 4: Insights & Interpretation

Insight 1: Manhattan Generates 87% of All Pickups

Derivation: GET /api/summary/by-borough returns trip counts grouped by borough. Dividing Manhattan's count (2,194,436) by the total clean record count (2,521,705) gives 87%.

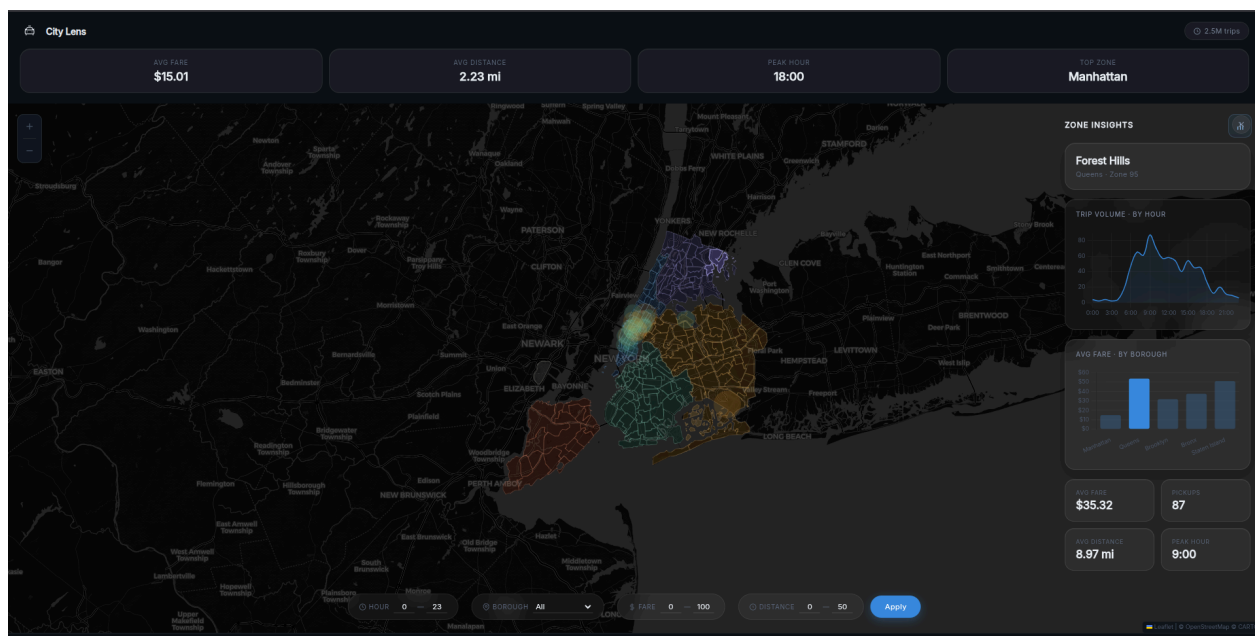
Interpretation: There are a lot of yellow taxis in Manhattan, New York City. The remaining 13% is split across Queens (10%), Brooklyn (2%), the Bronx (0.7%), and Staten Island (0.003%). Consistently high concentration is a result of Manhattan's commercial, hospitality and entertainment activity compared with the outer boroughs. It also sheds light on why yellow taxi service has always been most reliable in Manhattan, as supply will focus where demand is greatest.



Insight 2: Queens has the highest average fare, even though it has the lowest number of passengers.

Derivation: GET /api/summary/by-borough averages fare per borough. Although less than 12% of Manhattan's trips occur in Queens, the average fare in Queens is \$53.75, which is more than three times as high as the average fare in Manhattan of \$15.01.

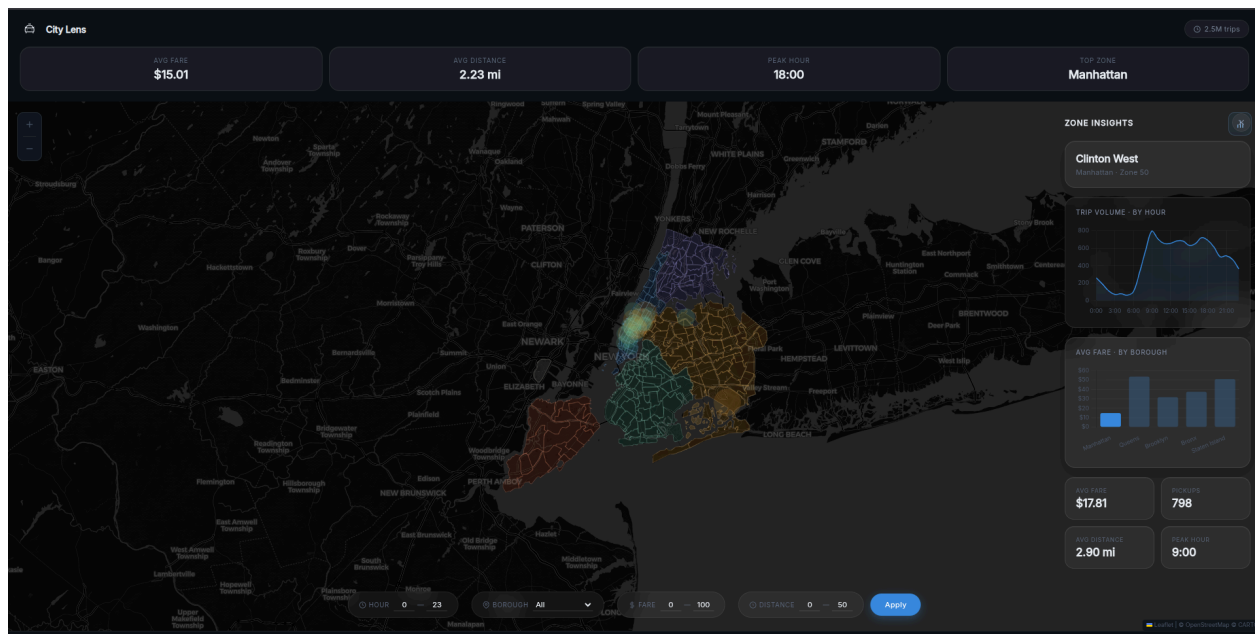
Interpretation: JFK Airport is the top pick-up location for the Queens figure (141,266 trips) while the top drop-off location is also in Queens (Long Island City with 14,696 trips). The average includes flat rate and toll charges in the airport, which greatly overestimate the average. This is an unmistakable two-line taxi service - short, cheap and frequent in Manhattan, and long, expensive in the airport. The opposite is true in a fare-per-mile analysis, though (\$11.53/mile versus the Bronx's \$7.84/mile), meaning that trips to Manhattan are more costly per unit of distance compared to outer borough trips.



Insight 3: Time of day for trips peaks at 18:00 (6 PM) and falls off significantly after midnight.

Derivation: GET /api/summary/by-hour will return trips by hour of the day. The highest number (around 120,000 trips) is during hour 18 and the lowest number (around 12,000 trips) is during hour 4.

Interpretation: The 6 PM peak is perfectly aligned with the end of the “typical” Manhattan workday, and that feature is closely connected to bus rides to transit hubs, restaurants and homes. This is a dramatic decrease after midnight, which can be attributed to both low demand and the fact that other modes of transportation (subway, rideshare) are available. Interestingly, the data reveals a secondary peak from 8-9am, (when people are commuting home from work), that is smaller than the evening peak, indicating that more people taxi home than to work, probably because people have more flexibility in the morning to use the subway.



Section 5: Reflection & Future Work

Technical Challenges

The most time consuming tech challenge was setting up Docker and PostgreSQL. This required some time to set up the database before it was available and reliable, because of port conflicts with an existing local PostgreSQL installation, mismatches in the socket path between the Docker daemon and the Docker client, and not having a user group. The solution was to bind the port 5432 within the container to the host port 5433, thus avoiding the need to remove the local installation.

Another interesting quirk was that the Python multiprocessing module creates new processes not attached to the memory of the parent process, in a parallel pipeline. The worker processes did not update the exclusion log list specified in the parent process. This was addressed by returning each worker's return value as a tuple (cleaned_chunk, local_log) and aggregating them once all workers have returned.

Another challenge was the path resolution of the .env file. The require('dotenv').config() in Node.js is based on the current working directory, not the file's location. This resulted in the environment variables not being loaded when the server was started from another directory, without any error message. The solution was to make an absolute path using path.join(__dirname, ...).

Team Challenges

The team worked with only two people working diligently on the backend, while the frontend was worked on at the same time. When designing API contracts, it was important that we agreed on endpoints, response formats, and field naming conventions before the front end was hooked up, which required effective communication to prevent integration issues.

Future Improvements

Multi-month dataset: One month of data is being used in the current deployment. A full year would show seasonal trends - holiday travel in December, dips in the summer and New Years Eve peaks, which cannot be seen in a month's snapshot.

The NYC TLC releases trip information once a month (monthly data). A production version could poll for new monthly releases and automatically add these to the database so that insights are always up-to-date.

Zone-level heatmap: This is the current heatmap with heat blobs displayed for hardcoded coordinates in the top 10 zones. It would be a more accurate implementation to calculate trip density for each of the 265 zones, and then colour the GeoJSON polygons accordingly — i.e., make a proper choropleth map instead of blobs.

Per-zone filtering: At the moment, the filter bar is filtering the trips table and refreshing the heat blobs. A better way to implement this would be to click a zone and view the effects of applying filters to it, such as the hour of day or day of week variations in traffic from JFK Airport.

Predictive model: If enough historical data were available, a simple regression model could be used to predict trip volume for a specific zone, hour and day of week. This would shift the dashboard from descriptive (what has happened) to predictive (what will happen), a worthwhile step toward having a real urban planning tool.